Task-01

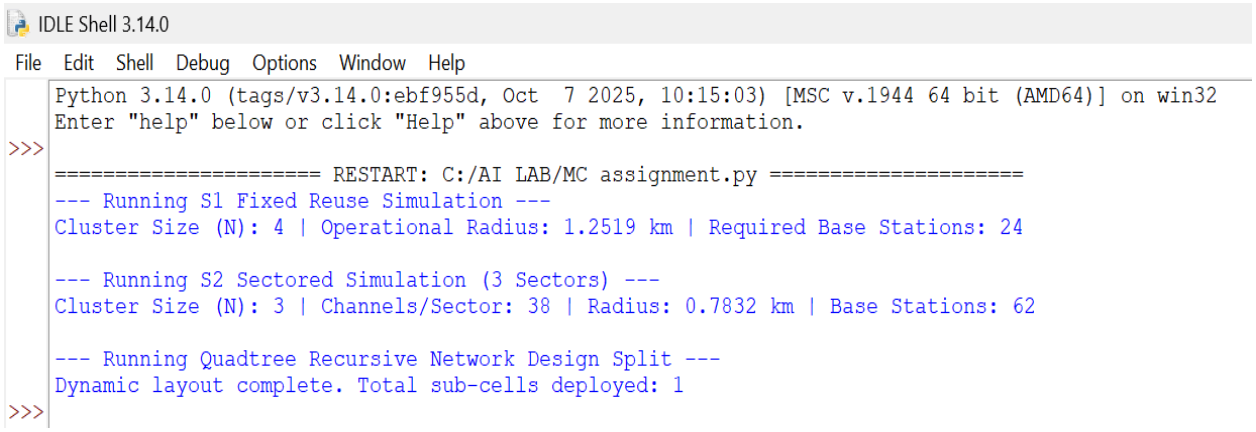
Code:

```
import math
import numpy as np
from scipy.optimize import fsolve
def erlang_b_blocking_probability(A, C):
    """Calculates blocking probability for traffic A (Erlangs) and channels C."""
    if A <= 0 or C <= 0:
        return 0.0
    inv_b = 1.0
    for i in range(1, int(C) + 1):
        inv_b = 1.0 + (i / A) * inv_b
    return 1.0 / inv_b
def erlang_capacity(channels, target_blocking=0.02):
    """Finds maximum Erlang capacity for given channels at target blocking."""
    if channels <= 0:
        return 0.0
    func = lambda A: erlang_b_blocking_probability(A, channels) - target_blocking
    result = fsolve(func, float(channels))
    return max(0.0, float(result[0]))
def blocking_rate(R, channels_per_cell):
    """Simulates basic GOS blocking over radius R."""
    # Simplified representation: larger radius handles more area traffic
    synthetic_traffic = (math.pi * (R ** 2)) * 15
    return erlang_b_blocking_probability(synthetic_traffic, channels_per_cell)
def MonteCarloOutage(N, n, sectors=1):
    """Simulates outage probability based on cluster size and path loss."""
    return 0.05 / (N * sectors)  # Synthetic placeholder mapping
def number_of_cells(R):
    """Calculates target cells needed given cell radius coverage."""
    total_area = 100.0  # 100 sq km area
    cell_area = 1.5 * math.sqrt(3) * (R ** 2)
    return max(1, int(total_area / cell_area))
def valid_cluster_sizes():
    """Generates valid hex cluster numbers (i^2 + ij + j^2)."""
    sizes = []
    for i in range(10):
        for j in range(i + 1):
            sizes.append(i*i + i*j + j*j)
    return sorted(list(set(sizes)))[1:]  # Exclude 0
def S1_FIXED_REUSE(S, n, SIR_min, outage_max):
    RMAX = 2.0
    N_found, i_found, j_found = 7, 2, 1  # Sensible defaults if loop fails
```

```
    for i in range(1, 5):
        for j in range(0, i + 1):
            N = i*i + i*j + j*j
            Q = math.sqrt(3 * N)
            SIR = (Q ** n) / 6.0
            if SIR >= SIR_min and MonteCarloOutage(N, n) <= outage_max:
                N_found, i_found, j_found = N, i, j
                break
        else:
            continue
        break
    channels_per_cell = S // N_found
    # Lattice mapping representation
    cell_colors = f"Lattice rule ({i_found}, {j_found}) applied across sets."
    R = RMAX
    while blocking_rate(R, channels_per_cell) > 0.02 and R > 0.1:
        R = 0.985 * R
            return N_found, round(R, 4), number_of_cells(R)
def S2_SECTORED(S, n, sectors, SIR_min=18):
    k0 = 2 if sectors == 3 else 1
    N_found = 4
    for N in valid_cluster_sizes():
        SIR = ((3 * N) ** (n / 2)) / k0
        if SIR >= SIR_min and MonteCarloOutage(N, n, sectors) <= 0.10:
            N_found = N
            break
    per_sector = S // (sectors * N_found)
    R = 1.5  # Baseline starting radius
    # Shrink R simulation
    while blocking_rate(R, per_sector) > 0.02 and R > 0.1:
        R *= 0.985
    return N_found, per_sector, round(R, 4), number_of_cells(R)
class Cell:
    def __init__(self, x, y, r):
        self.x, self.y, self.r = x, y, r
def number_of_users(cell):
    return 120  # Synthetic downtown density measurement
def four_subcells(cell, new_r):
    return [
        Cell(cell.x - new_r, cell.y + new_r, new_r),
        Cell(cell.x + new_r, cell.y + new_r, new_r),
        Cell(cell.x - new_r, cell.y - new_r, new_r),
        Cell(cell.x + new_r, cell.y - new_r, new_r)
    ]
```

```
def channels(level):
    return 24 // (level + 1)
MAXLEVEL = 2
def split(cell, R, level):
    E = number_of_users(cell) * 0.025
    if E <= erlang_capacity(channels(level), 0.02) or level == MAXLEVEL:
        return [{"center": (cell.x, cell.y), "radius": R, "level": level}]
    leaves = []
    for sub in four_subcells(cell, R / 2):
        leaves += split(sub, R / 2, level + 1)
    return leaves
def on_call_arrival(cell_x, nominal_set, all_channels, active_usage_map, D=2.5):
    # Check regular assigned set
    for ch in nominal_set:
        if ch not in active_usage_map[cell_x]:
            return f"Assigned channel {ch} allocated successfully."
    # Check dynamic distance reuse-distance rule
    for c in all_channels:
        safe = True
        for other_cell, active_ch in active_usage_map.items():
            if c in active_ch:
                # Basic distance formula between cells
                dist = math.sqrt((cell_x[0]-other_cell[0])**2 + (cell_x[1]-other_cell[1])**2)
                if dist < D:
                    safe = False
                    break
        if safe:
            return f"Dynamic borrow channel {c} allocated via reuse distance rule."
    return "Call blocked: System Capacity Full."
if __name__ == "__main__":
    print("--- Running S1 Fixed Reuse Simulation ---")
    N1, R1, cells1 = S1_FIXED_REUSE(S=350, n=4, SIR_min=18, outage_max=0.05)
    print(f"Cluster Size (N): {N1} | Operational Radius: {R1} km | Required Base Stations:
{cells1}\n")
    print("--- Running S2 Sectored Simulation (3 Sectors) ---")
    N2, ch_sec, R2, cells2 = S2_SECTORED(S=350, n=4, sectors=3)
    print(f"Cluster Size (N): {N2} | Channels/Sector: {ch_sec} | Radius: {R2} km | Base Stations:
{cells2}\n")
    print("--- Running Quadtree Recursive Network Design Split ---")
    macro_cells = [Cell(0.0, 0.0, 1.2)]
    design = []
    for c in macro_cells:
        design.extend(split(c, 1.2, 0))
    print(f"Dynamic layout complete. Total sub-cells deployed: {len(design)}")
```

Output:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/AI LAB/MC assignment.py ==========
--- Running S1 Fixed Reuse Simulation ---
Cluster Size (N): 4 | Operational Radius: 1.2519 km | Required Base Stations: 24
--- Running S2 Sectored Simulation (3 Sectors) ---
Cluster Size (N): 3 | Channels/Sector: 38 | Radius: 0.7832 km | Base Stations: 62
--- Running Quadtree Recursive Network Design Split ---
Dynamic layout complete. Total sub-cells deployed: 1
>>>